

LAPORAN JOBSHEET 18: OPTIMIZE MATKUL PEMOGRAMAN BERBASIS NETWORK

AHMAD DZUL FADHLI HANNAN | TI3F | 2341720106

PRAKTIKUM 1 – IMAGE OPTIMIZATION

A. Optimasi Gambar Lokal (Public Folder)

Studi Kasus:

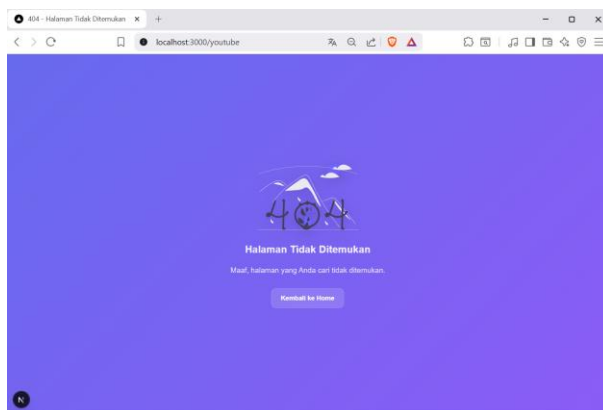
Mengganti tag pada halaman 404 dengan next/image. Langkah:

Buka file src/pages/404.tsx

Modifikasi line 7 menjadi line 8-11

```
pemograman_framework - 404.tsx
1 import styles from "@styles/404.module.scss";
2 import Link from "next/dist/client/link";
3 import Image from "next/image";
4 const Custom404 = () => {
5   return (
6     <div className={styles.error}>
7       <head>
8         <title>404 - Halaman Tidak Ditemukan</title>
9       </head>
10      
11      <image
12        src="/page-not-found.svg"
13        alt="404"
14        width={400}
15        height={200}
16        className={styles.error__image}
17      />
18      <h2 className={styles.error_subtitle}>
19        Halaman Tidak Ditemukan
20      </h2>
21      <p className={styles.error_desc}>Maaf, halaman yang Anda cari tidak ditemukan.</p>
22      <Link href="/" className={styles.error_button}>Kembali ke Home</Link>
23    </div>
24  );
25 };
26 export default Custom404;
```

Hasil:



- Warning hilang
- Image dioptimasi otomatis
- Mengurangi bandwidth

- Mendukung lazy loading otomatis

B. Optimasi Gambar Remote (External URL)

Buka file views/product/index.tsx

Modifikasi file index.tsx

```
pemograman_framework - index.tsx
3 import Image from "next/image";
```

```
pemograman_framework - index.tsx
14 {<img src={products_image} alt={products_name} className={styles.product_content_image} width={200} height={200} />}
15 {<img src={products_image} alt={products_name} className={styles.product_content_image} width={200} height={200} />}
16 {<img src={products_image} alt={products_name} className={styles.product_content_image} width={200} height={200} />}
```

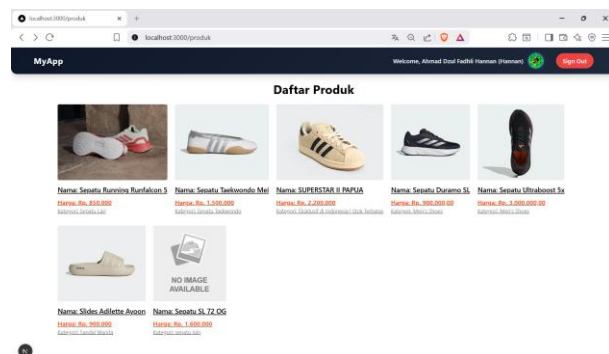
Diset length > 10 karena ada data yg isinya cuma http sehingga muncul error karena image tidak terender.

Note : dikarenakan gambar diambil dari url tertentu maka konfigurasi berbeda

Buka file next.config.js

```
pemograman_framework - next.config.js
1 /** @type {import('next').NextConfig} */
2 const nextConfig = {
3   reactStrictMode: true,
4   images: {
5     remotePatterns: [
6       {
7         protocol: 'https',
8         hostname: 'assets.adidas.com',
9         port: "",
10        pathname: "/*",
11      }
12    ],
13  },
14 }
15
16 module.exports = nextConfig
```

Hasil:



- Gambar di-proxy melalui /_next/image
- Performa lebih optimal
- Kompresi otomatis

PRAKTIKUM 2 – FONT OPTIMIZATION

A. Menggunakan next/font

Buka file index.tsx pada folder Appshell/index.tsx dan modifikasi

```

3 import { Roboto } from "next/font/google";

```

```

11 const roboto = Roboto({
12   subsets: ['latin'],
13   weight: ['400', '500', '700'],
14 });

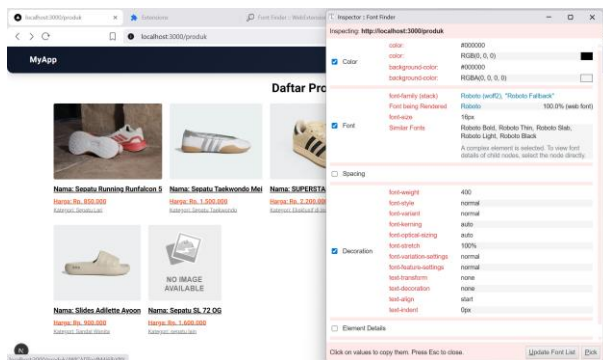
```

```

23 <main className={roboto.className}>

```

Jalankan browser localhost:3000/produk maka font akan berubah menjadi roboto untuk mengecek fontnya bisa menggunakan extension FontFinder



Hasil:

- Tidak perlu load dari CDN manual
- Tidak blocking render

- Performance meningkat
- Tidak terjadi FOUT (Flash of Unstyled Text)

PRAKTIKUM 3 – SCRIPT OPTIMIZATION

B. Menggunakan next/script

Buka file index.tsx pada folder layouts/Navbar dan modifikasi

```
pemograman_framework - index.tsx
1  import Script from "next/dist/client/script";
```

```
pemograman_framework - index.tsx
13  {/* <div className={styles.navbar__brand}>
14    MyApp
15  </div> */}
16
17  <div className={styles.navbar__brand} id="title"></div>
18  <Script id="title-script" strategy="lazyOnload">
19    {`document.getElementById('title').innerHTML='MyApp';`}
20  </Script>
```

Pada kasus diatas kita merubah line 11 menggunakan model Typescript dapat terlihat ketika kita refresh web produk tulisan myApp akan terlihat berkedip

Perbedaan mendasar antara Line 11-13 dan Line 15-18 pada file index.tsx Anda terletak pada metode rendering teks dan interaksi dengan DOM (Document Object Model). Berikut adalah rincian perbedaannya:

I. Metode Rendering

- Line 11-13 (Standard React/JSX): Ini adalah cara standar React. Teks "MyApp" ditulis langsung di dalam tag div. React akan langsung merender teks ini ke dalam HTML saat komponen dimuat.
- Line 15-18 (Next.js Script & Manual DOM): Menggunakan komponen `<Script>` dari Next.js dengan strategi `lazyOnload`. Teks tidak ada di dalam HTML saat awal dimuat, melainkan baru "disuntikkan" secara manual menggunakan perintah JavaScript `document.getElementById('title').innerHTML = 'MyApp';` setelah script tersebut diunduh di latar belakang.

II. Performa dan SEO

- Line 11-13: Sangat baik untuk SEO karena teks "MyApp" langsung terbaca oleh robot pencari (crawler) dalam kode sumber HTML.
- Line 15-18: Kurang baik untuk SEO untuk konten penting karena teks baru muncul setelah JavaScript dijalankan. Strategi `lazyOnload` berarti script ini dijalankan paling

akhir setelah semua sumber daya utama selesai dimuat, sehingga mungkin ada jeda waktu (delay) sebelum teks muncul di layar.

III. Keamanan (XSS)

- a. Line 11-13: Aman karena React secara otomatis melakukan escape pada string untuk mencegah serangan Cross-Site Scripting (XSS).
- b. Line 15-18: Memiliki risiko keamanan lebih tinggi karena menggunakan `.innerHTML`. Jika data yang dimasukkan berasal dari input user, ini bisa dimanfaatkan untuk menyuntikkan skrip berbahaya.

C. Strategi Script

Strategy	Fungsi
beforeInteractive	Sebelum halaman interaktif
afterInteractive	Setelah halaman interaktif
lazyOnload	Setelah semua selesai
worker	Web worker

Hasil:

- Script tidak blocking
- Cocok untuk Google Analytics
- Performa lebih ringan

PRAKTIKUM 4 – OPTIMASI AVATAR DENGAN NEXT/IMAGE

Buka file `index.tsx` pada folder `layouts/navbar` dan modifikasi :

```
pemograman_framework - index.tsx
1  import Script from "next/dist/client/script";
```

```
pemograman_framework - index.tsx
27  {data.user.image && (
28    <Image
29      src={data.user.image}
30      alt={data.user.fullname}
31      className={styles.navbar__user__image}
32      width={50}
33      height={50}
34    />
35  )}
```

Tambahkan hostname Google dan Github:

```
pemograman_framework - next.config.js

1  /** @type {import('next').NextConfig} */
2  const nextConfig = {
3    reactStrictMode: true,
4    images: {
5      remotePatterns: [
6        {
7          protocol: 'https',
8          hostname: 'assets.adidas.com',
9          port: "",
10         pathname: "/*",
11       },
12       {
13         protocol: 'https',
14         hostname: 'lh3.googleusercontent.com',
15         port: "",
16         pathname: "/*",
17       },
18       {
19         protocol: 'https',
20         hostname: 'avatars.githubusercontent.com',
21         port: "",
22         pathname: "/*",
23       }
24     ],
25   },
26 }
27
28 module.exports = nextConfig
```

TUGAS PRAKTIKUM

1. Optimasi semua image di project menggunakan next/image

1. src/components/layouts/navbar/index.tsx

```
pemograman_framework - index.tsx

27 {data.user.image && (
28   <Image
29     src={data.user.image}
30     alt={data.user.fullname}
31     className={styles.navbar__user__image}
32     width={50}
33     height={50}
34   />
```

2. src/views/lapar/index.tsx

```
pemograman_framework - index.tsx

21 <Image src={lapar_image.length > 50 ? lapar_image : "/no-image.svg"} alt={lapar.name} className={styles.lapar_content_item_image} width={300} height={300} />
```

3. src/views/DetailProduct/index.tsx

```
pemograman_framework - index.tsx

21 <Image src={imageSrc} alt={products.name} className={styles.produkdetail_image} width={300} height={300} />
```

4. src/views/produk/main.tsx

```
pemograman_framework - main.tsx
11 <Image src="/shopping.svg" alt="shopping" className="w-80" width={320} height={320} />
```

5. src/views/lapar/lapar/main.tsx

```
pemograman_framework - main.tsx
11 <Image src="/shopping.svg" alt="shopping" className="w-80" width={320} height={320} />
```

6. src/pages/404.tsx

```
pemograman_framework - 404.tsx
11 <Image
12   src="/page-not-found.svg"
13   alt="404"
14   width={400}
15   height={200}
16   className={styles.error__image}
17 />
```

7. src/views/product/index.tsx

```
pemograman_framework - index.tsx
20 <Image src={products_image.length > 10 ? products_image : "/no-image.svg"} alt={products_name} className={styles.product_content_image} width={200} height={200} />
```

2. Gunakan minimal 1 font dari next/font

/Appshell/index.tsx

```
pemograman_framework - index.tsx
1 import { useRouter } from "next/router";
2 import Navbar from "../navbar";
3 import { Roboto } from "next/font/google";
4
5 const disableNavbar = ['/auth/login', '/auth/register', '/404'];
6
7 type AppShellProps = {
8   children: React.ReactNode;
9 }
10
11 const roboto = Roboto({
12   subsets: ['latin'],
13   weight: ['400', '500', '700'],
14 });
15
16 const AppShell = (props: AppShellProps) => {
17   const {children} = props;
18   const {pathname} = useRouter();
19   // Insight: Gunakan router.pathname jika ada query parameter untuk memastikan path yang tepat
20   // const router = useRouter();
21   // console.log(router);
22   return (
23     <main className={roboto.className}>
24       {disableNavbar.includes(pathname) && <Navbar />}
25       {children}
26     </main>
27   );
28 };
29
30 export default AppShell;
```

Menggunakan Roboto

3. Tambahkan script Google Analytics menggunakan next/script

Tambahkan /components/analytics/Analytics.tsx

```

1 import Script from "next/script";
2 import { useEffect } from "react";
3 import { useRouter } from "next/router";
4
5 const gaId = process.env.NEXT_PUBLIC_GA_ID;
6
7 declare global {
8   interface Window {
9     gtag?: (...args: unknown[]) => void;
10   }
11 }
12
13 const Analytics = () => {
14   const router = useRouter();
15
16   useEffect(() => {
17     if (!gaId) return;
18
19     const handleRouteChange = (url: string) => {
20       if (typeof window.gtag !== "function") return;
21       window.gtag("config", gaId, {
22         page_path: url,
23       });
24     };
25
26     router.events.on("routeChangeComplete", handleRouteChange);
27     return () => {
28       router.events.off("routeChangeComplete", handleRouteChange);
29     };
30   }, [router.events]);
31
32   if (!gaId) return null;
33
34   return (
35     <Script
36       src="https://www.googletagmanager.com/gtag/js?id=${gaId}"
37       strategy="afterInteractive"
38     />
39     <Script id="google-analytics" strategy="afterInteractive">
40       {`
41         window.dataLayer = window.dataLayer || [];
42         function gtag(){dataLayer.push(arguments);}
43         gtag('js', new Date());
44         gtag('config', "${gaId}", { page_path: window.location.pathname });
45       `}
46     </Script>
47   )
48 }
49
50 export default Analytics;

```

4. Terapkan dynamic import pada minimal 1 komponen

```

1 import "../styles/globals.css";
2 import type { AppProps } from "next/app";
3 import AppShell from "@components/layouts/AppShell";
4 import { SessionProvider } from "next-auth/react";
5 import dynamic from "next/dynamic";
6
7 const Analytics = dynamic(() => import("@components/analytics/Analytics"), { ssr: false });
8
9 export default function App({ Component, pageProps: { session, ...pageProps } }: AppProps) {
10   return (
11     <SessionProvider session={pageProps.session}>
12       <Analytics />
13       <AppShell>
14         <Component {...pageProps} />
15       </AppShell>
16     </SessionProvider>
17   );
18 }
19
20

```

5. Dokumentasikan perubahan performa (screenshot Lighthouse)

Note: Untuk perbandingan agar adil:

- pada branch pertemuan-6 sebagai patokan kondisi sebelum optimasi dan branch pertemuan-7 sebagai patokan kondisi setelah optimasi,
- semua ekstensi browser menyala,
- route ke /, dan /produk,
- untuk platform yg digunakan adalah desktop.
- Pada env.local NEXT_PUBLIC_GA_ID= dihapus

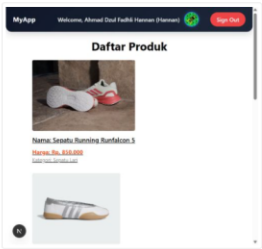
Sebelum



Performance

Values are estimated and may vary. The [performance score is calculated](#) directly from these metrics. [See calculator.](#)

▲ 0-49 ■ 50-89 ● 90-100



METRICS

Expand view

● First Contentful Paint
0.3 s

■ Largest Contentful Paint
1.4 s

● Total Blocking Time
0 ms

● Cumulative Layout Shift
0.049

● Speed Index
1.0 s

🗲 View Treemap



Show audits relevant to: [All](#) [FCP](#) [LCP](#) [CLS](#)

INSIGHTS

- ▲ Improve image delivery — Est savings of 136 KiB
- ▲ Legacy JavaScript — Est savings of 11 KiB
- ▲ LCP request discovery
- Use efficient cache lifetimes — Est savings of 1 KiB
- Layout shift culprits
- LCP breakdown
- 3rd parties

These insights are also available in the Chrome DevTools Performance Panel - [record a trace](#) to view more detailed information.

DIAGNOSTICS

- ▲ Minify JavaScript — Est savings of 188 KiB
- ▲ Reduce unused JavaScript — Est savings of 558 KiB
- ▲ Page prevented back/forward cache restoration — 1 failure reason
- User Timing marks and measures — 24 user timings

More information about the performance of your application. These numbers don't [directly affect](#) the Performance score.

PASSED AUDITS (18)

Show

Captured at Apr 11, 2026, 10:06 AM GMT+7

Emulated Desktop with Lighthouse 13.0.2

Single page session

Initial page load

Custom throttling

Using Chromium 146.0.0.0 with devtools

Sesudah

96

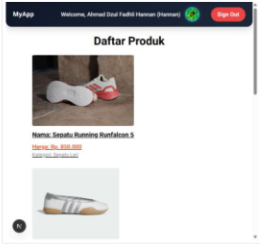
Performance

Values are estimated and may vary. The [performance score is calculated](#) directly from these metrics. [See calculator.](#)

▲ 0-49

■ 50-89

● 90-100



METRICS

Expand view

● First Contentful Paint

0.3 s

● Total Blocking Time

0 ms

● Speed Index

1.1 s

■ Largest Contentful Paint

1.3 s

● Cumulative Layout Shift

0.048

 View Treemap



INSIGHTS

These insights are also available in the Chrome DevTools Performance Panel - [record a trace](#) to view more detailed information.

▲ LCP request discovery

■ Use efficient cache lifetimes — Est savings of 1 KiB

■ Legacy JavaScript — Est savings of 11 KiB

○ Layout shift culprits

○ LCP breakdown

○ 3rd parties

DIAGNOSTICS

More information about the performance of your application. These numbers don't [directly affect](#) the Performance score.

▲ Minify JavaScript — Est savings of 196 KiB

▲ Reduce unused JavaScript — Est savings of 333 KiB

▲ Page prevented back/forward cache restoration — 1 failure reason

○ User Timing marks and measures — 26 user timings

○ Avoid long main-thread tasks — 1 long task found

PASSED AUDITS (18)

Show

 Captured at Apr 11, 2026, 10:04 AM GMT+7

 Emulated Desktop with Lighthouse 13.0.2

 Single page session

 Initial page load

 Custom throttling

 Using Chromium 146.0.0.0 with devtools

Generated by **Lighthouse 13.0.2** | [File an issue](#)

Kesimpulan

Optimasi yang dilakukan berhasil meningkatkan performa halaman /produk, meskipun peningkatannya tidak terlalu besar. Skor Lighthouse naik dari 95 menjadi 96.

Beberapa metrik menunjukkan perbaikan:

- LCP turun dari 1.4s menjadi 1.3s
- TTI turun dari 1.4s menjadi 1.3s
- CLS sedikit lebih stabil, dari 0.049 menjadi 0.048
- TBT tetap sangat baik di angka 0 ms

REFLEKSI & DISKUSI

1. Mengapa biasa tidak optimal?

Karena gambar yang tidak terkompresi dengan baik atau tidak menggunakan teknik seperti lazy loading dapat memperlambat waktu muat halaman.

2. Apa perbedaan font CDN dan next/font?

Font CDN mengambil file font dari server eksternal saat runtime, sedangkan next/font mengelola, mengoptimasi, dan menyajikan font langsung dari aplikasi agar lebih cepat dan konsisten.

3. Mengapa script bisa membuat website lambat?

Karena menambah beban download, parsing, dan eksekusi JavaScript di main thread saat halaman dimuat.

4. Kapan harus menggunakan dynamic import?

Saat komponen atau library tidak dibutuhkan di initial render agar bundle awal lebih kecil dan waktu muat awal lebih cepat.

5. Apa dampak bundle size terhadap UX?

Bundle size yang besar memperburuk UX karena meningkatkan waktu loading, menghambat interaksi awal, dan membuat aplikasi terasa lambat terutama di perangkat atau jaringan lemah.

KESIMPULAN

Dalam praktikum ini mahasiswa telah mempelajari:

- ✓ Image Optimization
- ✓ Remote Image Configuration
- ✓ Font Optimization
- ✓ Script Optimization
- ✓ Dynamic Import
- ✓ Lazy Loading

Semua fitur ini merupakan keunggulan utama Next.js dalam meningkatkan performa aplikasi modern.

LAPORAN JOBSHEET 19: UNIT TESTING MATKUL PEMOGRAMAN BERBASIS NETWORK

AHMAD DZUL FADHLI HANNAN | TI3F | 2341720106

PRAKTIKUM 1 – SETUP JEST DI NEXT.JS

Langkah 1- Install Dependencies

Jalankan:

```
npm install jest jest-environment-jsdom @testing-library/react @testing-library/jest-dom --save-dev --force
```

```
> npm install jest jest-environment-jsdom @testing-library/react @testing-library/jest-dom --save-dev --force
npm warn using --force recommended protection disabled.
npm warn deprecated inflight@1.0.5: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce
async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated whatwg-url@7.1.0: The WHATWG/bytes library for a more spec-compliant and faster implementation
npm warn deprecated glob@7.1.6: Glob versions 7.x are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current
version. Please update. Support for old versions may be purchased (at merchant rates) by contacting i@npm.js
npm warn deprecated glob@7.1.6: Glob versions 7.x are not supported, and contain widely publicized security vulnerabilities, which have been fixed in the current
version. Please update. Support for old versions may be purchased (at merchant rates) by contacting i@npm.js
added 286 packages, and audited 776 packages in 10s
195 packages are looking for funding
  run `npm fund` for details
4 vulnerabilities (1 moderate, 3 high)
  to address all issues, run:
    npm audit fix
  or, to address a subset of these issues, run:
    npm audit -- --only=moderate
  Run `npm audit` for details.
```

Langkah 2 - Buat File Konfigurasi

Doc : <https://nextjs.org/docs/pages/guides/testing/jest>

Buat file:

o jest.config.mjs

Isi dengan:

```
pemograman_framework - jest.config.mjs

1  import nextJest from 'next/jest.js';
2
3  const createJestConfig = nextJest({
4    dir: './',
5  })
6
7  const config = {
8    coverageProvider: 'v8',
9    testEnvironment: 'jsdom',
10 }
11
12 export default createJestConfig(config);
```

Langkah 3 - Tambahkan Script di package.json

```
pemograman_framework - package.json

10 "test": "jest --passWithNoTests -u",
11 "test:coverage": "npm run test -- --coverage",
12 "test:watch": "jest --watch -u"
```

PRAKTIKUM 2 – STRUKTUR FOLDER TESTING

Buat folder:

src/___test___/

Struktur contoh:

src

|—— pages

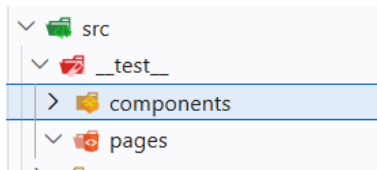
|—— components

|—— views

|—— ___test___

|—— pages

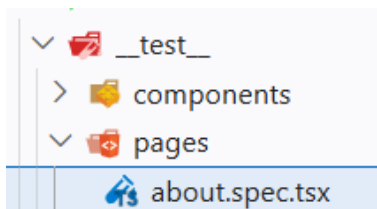
|—— components



PRAKTIKUM 3 – TESTING HALAMAN ABOUT

Langkah 1 - Buat File Testing

src/___test___/pages/about.spec.tsx



Langkah 2 - Contoh Testing Snapshot. Pada `about.spec.tsx` tambahkan code berikut :

```
pemograman_framework - about.spec.tsx

1 import { render } from '@testing-library/react';
2 import AboutPage from "@pages/about";
3
4 describe("AboutPage", () => {
5     it("renders about page correctly", () => {
6         const page = render(<AboutPage />);
7         expect(page).toMatchSnapshot();
8     });
9 })
10
```

Langkah 3 - Jalankan Testing

- `npm run test`
- Jika berhasil:
- PASS `about.spec.tsx`

```
> npm run test

> my-app@0.1.0 test
> jest --passWithNoTests -u

PASS src/__test__/pages/about.spec.tsx
  AboutPage
    ✓ renders about page correctly (20 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:  1 passed, 1 total
Time:        0.815 s, estimated 1 s
Ran all test suites.
```

PRAKTIKUM 4 – COVERAGE REPORT

Jalankan:

o `npm run test:coverage`

```
> jest --passWithNoTests -u --coverage

PASS src/__test__/pages/about.spec.tsx
  AboutPage
    ✓ renders about page correctly (21 ms)

-----|-----|-----|-----|-----|
File    | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|
All files|     100 |     100 |     100 |     100 |
index.tsx|     100 |     100 |     100 |     100 |
-----|-----|-----|-----|-----|

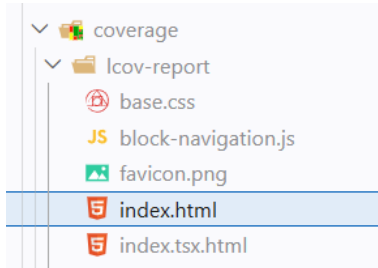
Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:  1 passed, 1 total
Time:        1.673 s
Ran all test suites.
```

Akan muncul folder:

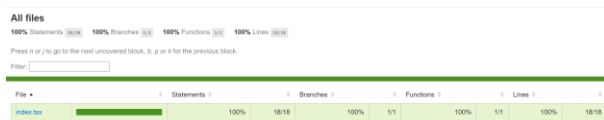
o coverage/

o Buka:

- coverage/lcov-report/index.html (buka di melalui explorer)



Target perusahaan biasanya:



Minimum 80% coverage

PRAKTIKUM 5 – KONFIGURASI COVERAGE LENGKAP

Update jest.config.mjs:


```

● ● ● pemograman_framework - jest.config.mjs

1  import nextJest from 'next/jest.js';
2
3  const createJestConfig = nextJest({
4    dir: './',
5  })
6
7  const config = {
8    coverageProvider: 'v8',
9    testEnvironment: 'jsdom',
10   modulePaths: ['<rootDir>/src/'],
11   collectCoverage: true,
12   collectCoverageFrom: [
13     '**/*.{ts,tsx}',
14     '**/*.d.ts',
15     '!**/node_modules/**',
16     '!**/.next/**',
17     '!**/coverage/**',
18     '!**/jest.config.mjs',
19     '!**/next.config.mjs',
20     '!**/types/**',
21     '!**/views/**',
22     '!**/pages/api/**',
23   ],
24 }
25
26 export default createJestConfig(config);

```

Jalankan npm run test:coverage

```

> npm run test:coverage

```

index.tsx	0	0	0	0	1-30
my-app/src/components/layouts/navbar	0	0	0	0	1-55
index.tsx	0	0	0	0	1-48
my-app/src/middleware	0	0	0	0	1-26
withAuth.ts	0	0	0	0	1-19
my-app/src/pages	0	0	0	0	1-13
404.tsx	0	0	0	0	1-21
_app.tsx	0	0	0	0	1-100
_document.tsx	0	0	0	0	1-100
index.tsx	0	0	0	0	1-100
my-app/src/pages/about	100	100	100	100	1-16
index.tsx	100	100	100	100	1-15
my-app/src/pages/admin	0	0	0	0	1-9
index.tsx	0	0	0	0	1-18
my-app/src/pages/blog	0	0	0	0	1-13
[slug].tsx	0	0	0	0	1-13
index.tsx	0	0	0	0	1-17
my-app/src/pages/category	0	0	0	0	1-31
[...slug].tsx	0	0	0	0	1-25
my-app/src/pages/editor	0	0	0	0	1-28
index.tsx	0	0	0	0	1-34
my-app/src/pages/lapar	0.98	0	0	0.98	1-20
[id].tsx	0	0	0	0	1-29
index.tsx	3.12	0	0	3.12	1-50
server.tsx	0	0	0	0	1-14
static.tsx	0	0	0	0	1-13
my-app/src/pages/produk	0	0	0	0	1-9
index.tsx	0	0	0	0	1-15
server.tsx	0	0	0	0	1-9
static.tsx	0	0	0	0	1-9
my-app/src/pages/produk/[produk]	1	0	0	1	1-19
index.tsx	0	0	0	0	1-139
server.tsx	0	0	0	0	1-2
static.tsx	1.96	0	0	1.96	
my-app/src/pages/profil	0	0	0	0	
edit.tsx	0	0	0	0	
index.tsx	0	0	0	0	
my-app/src/pages/setting	0	0	0	0	
app.tsx	0	0	0	0	
my-app/src/pages/shop	0	0	0	0	
[...slug].tsx	0	0	0	0	
my-app/src/pages/user	0	0	0	0	
index.tsx	0	0	0	0	
my-app/src/pages/user/password	0	0	0	0	
index.tsx	0	0	0	0	
my-app/src/utlis	0	0	0	0	
firebase.ts	0	0	0	0	
my-app/src/utlis/db	0	0	0	0	
service/firebase.ts	0	0	0	0	
my-app/src/utlis/sur	0	0	0	0	
fetcher.ts	0	0	0	0	

```

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:  1 passed, 1 total
Time:        1.664 s
Ran all test suites.

```

Jika dilihat di index.htmlnya

All files
2.21% Statements (36/166) 2.85% Branches (1/35) 2.85% Functions (1/35) 2.21% Lines (36/166)

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
my-app	0%	0%	0%	0%
my-app/src	0%	0%	0%	0%
my-app/src/components/analytic	0%	0%	0%	0%
my-app/src/components/layout/appshell	0%	0%	0%	0%
my-app/src/components/layout/header	0%	0%	0%	0%
my-app/src/middleware	0%	0%	0%	0%
my-app/src/pages	0%	0%	0%	0%
my-app/src/pages/about	100%	100%	100%	100%
my-app/src/pages/admin	0%	0%	0%	0%
my-app/src/pages/blog	0%	0%	0%	0%
my-app/src/pages/category	0%	0%	0%	0%
my-app/src/pages/editor	0%	0%	0%	0%
my-app/src/pages/home	0.98%	1/102	0%	0.98%

PRAKTIKUM 6 – TESTING DENGAN GETBYTESTID

Tambahkan pada About Page

- `<h1 data-testid="title">About Page</h1>`

```
pemograman_framework - index.tsx
11 <h1 data-testid="title">Ini Adalah Halaman About</h1>
```

Update Testing pada about.spec.tsx

```
pemograman_framework - about.spec.tsx
1 import { render } from '@testing-library/react';
2 import AboutPage from "@pages/about";
3
4 describe("AboutPage", () => {
5   it("renders about page correctly", () => {
6     const page = render(<AboutPage />);
7     expect(page.getByTestId("title").textContent).toBe("Ini Adalah Halaman About");
8     expect(page).toMatchSnapshot();
9   });
10 })
11
```

Dicoba untuk di run

```
> npm run test:coverage
-----
File                                     | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files                               |    2.21 |    2.85 |    2.85 |    2.21 |
my-app                                  |      0 |      0 |      0 |      0 |
  next-env.d.ts                         |      0 |      0 |      0 |      0 | 1-6
  my-app/src                             |      0 |      0 |      0 |      0 |
    middleware.ts                       |      0 |      0 |      0 |      0 | 1-27
    my-app/src/components/analytics     |      0 |      0 |      0 |      0 |
      Analytics.tsx                     |      0 |      0 |      0 |      0 | 1-52
    my-app/src/components/layouts/Appshell |      0 |      0 |      0 |      0 |
      index.tsx                         |      0 |      0 |      0 |      0 | 1-30
    my-app/src/components/layouts/navbar |      0 |      0 |      0 |      0 |
      index.tsx                         |      0 |      0 |      0 |      0 | 1-55
    my-app/src/middleware                |      0 |      0 |      0 |      0 |
      withAuth.ts                       |      0 |      0 |      0 |      0 | 1-48
    my-app/src/pages                     |      0 |      0 |      0 |      0 |
      404.tsx                           |      0 |      0 |      0 |      0 | 1-26
      _app.tsx                           |      0 |      0 |      0 |      0 | 1-19
      _document.tsx                     |      0 |      0 |      0 |      0 | 1-13
      index.tsx                         |      0 |      0 |      0 |      0 | 1-21
    my-app/src/pages/about               |    100 |    100 |    100 |    100 |
      index.tsx                         |    100 |    100 |    100 |    100 |
    my-app/src/pages/admin               |      0 |      0 |      0 |      0 |
      index.tsx                         |      0 |      0 |      0 |      0 | 1-16
    my-app/src/pages/blog                 |      0 |      0 |      0 |      0 |
      [slug].tsx                         |      0 |      0 |      0 |      0 | 1-15
      index.tsx                         |      0 |      0 |      0 |      0 | 1-9
    my-app/src/pages/category             |      0 |      0 |      0 |      0 |
      [...slug].tsx                     |      0 |      0 |      0 |      0 | 1-18
    my-app/src/pages/editor               |      0 |      0 |      0 |      0 |
      index.tsx                         |      0 |      0 |      0 |      0 | 1-13
    my-app/src/pages/lapar                |    0.98 |      0 |      0 |    0.98 |
      [id].tsx                           |      0 |      0 |      0 |      0 | 1-17
      index.tsx                         |    3.12 |      0 |      0 |    3.12 | 1-31
      server.tsx                         |      0 |      0 |      0 |      0 | 1-25
      static.tsx                         |      0 |      0 |      0 |      0 | 1-28
    my-app/src/pages/produk               |      0 |      0 |      0 |      0 |
      index.tsx                         |      0 |      0 |      0 |      0 | 1-34
      server.tsx                         |      0 |      0 |      0 |      0 | 1-25
      static.tsx                         |      0 |      0 |      0 |      0 | 1-28
    my-app/src/pages/produk/[produk]      |      1 |      0 |      1 |      1 |
      index.tsx                         |      0 |      0 |      0 |      0 | 1-20
      server.tsx                         |      0 |      0 |      0 |      0 | 1-29
      static.tsx                         |    1.96 |      0 |      0 |    1.96 | 1-50
    my-app/src/pages/profil               |      0 |      0 |      0 |      0 |
      edit.tsx                           |      0 |      0 |      0 |      0 | 1-14
      index.tsx                         |      0 |      0 |      0 |      0 | 1-13
    my-app/src/pages/setting              |      0 |      0 |      0 |      0 |
      app.tsx                           |      0 |      0 |      0 |      0 | 1-9
    my-app/src/pages/shop                 |      0 |      0 |      0 |      0 |
      [...slug].tsx                     |      0 |      0 |      0 |      0 | 1-15
    my-app/src/pages/user                 |      0 |      0 |      0 |      0 |
      index.tsx                         |      0 |      0 |      0 |      0 | 1-9
    my-app/src/pages/user/password        |      0 |      0 |      0 |      0 |
      index.tsx                         |      0 |      0 |      0 |      0 | 1-9
    my-app/src/utils                      |      0 |      0 |      0 |      0 |
      firebase.ts                       |      0 |      0 |      0 |      0 | 1-19
    my-app/src/utils/db                   |      0 |      0 |      0 |      0 |
      servicefirebase.ts                 |      0 |      0 |      0 |      0 | 1-139
    my-app/src/utils/sur                   |      0 |      0 |      0 |      0 |
      fetcher.ts                         |      0 |      0 |      0 |      0 | 1-2
-----
Snapshot Summary
> 1 snapshot updated from 1 test suite.

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:  1 updated, 1 total
Time:        1.73 s
Ran all test suites.
```

Coba Jika dibuat Salah:

- Rubah menjadi toBe("About")

```
pemograman_framework - about.spec.tsx

1  import { render } from '@testing-library/react';
2  import AboutPage from "@pages/about";
3
4  describe("AboutPage", () => {
5    it("renders about page correctly", () => {
6      const page = render(<AboutPage />);
7      expect(page.getByTestId("title").textContent).toBe("About");
8      expect(page).toMatchSnapshot();
9    });
10 })
11
```

Jalan kan dan Hasil:

o FAIL

Expected: "About"

Received: " Ini Adalah Halaman About"

```

> npm run test:coverage

> my-app@0.1.0 test:coverage
> npm run test -- --coverage

> my-app@0.1.0 test
> jest --passWithNoTests -u --coverage

FAIL src/__test__/pages/about.spec.tsx
  AboutPage
    ✕ renders about page correctly (26 ms)

  ● AboutPage › renders about page correctly

    expect(received).toBe(expected) // Object.is equality

    Expected: "About"
    Received: "Ini Adalah Halaman About"

       5 |     it("renders about page correctly", () => {
       6 |       const page = render(<AboutPage />);
     >    7 |       expect(page.getByTestId("title").textContent).toBe("About");
          |                                     ^
       8 |       expect(page).toMatchSnapshot();
       9 |     });
      10 | })
      at Object.toBe (src/__test__/pages/about.spec.tsx:7:55)

```

PRAKTIKUM 7 – TESTING PAGE DENGAN ROUTER (MOCKING)

Kita coba untuk melakukan testing pada halaman produk

Buat file `/__test__/pages/product.spec.tsx`

Tambahkan kode berikut

```

pemogramaran_framework - product.spec.tsx
1  import { render, screen } from '@testing-library/react';
2  import TampilanProduk from '@pages/produk';
3
4  describe("Product Page", () => {
5    it("renders product page correctly", () => {
6      const page = render(<TampilanProduk />);
7      expect(screen.getByTestId("title").textContent).toBe("Produk Page");
8      expect(page).toMatchSnapshot();
9    })
10 })

```

Ketika testing halaman Product, sering muncul error:

- NextRouter was not mounted

```

FAIL src/__test__/pages/product.spec.tsx
  ● Product Page › renders product page correctly

    NextRouter was not mounted. https://nextjs.org/docs/messages/next-router-not-mounted

      13 |     // }
      14 |     // }, {isLogin});
     > 15 |     const {push} = useRouter();
          |                       ^

```

Solusi: Mock Next Router

Tambahkan di file `product.spec.tsx`

```
pemograman_framework - product.spec.tsx

1 import { render, screen } from '@testing-library/react';
2 import TampilanProduk from './pages/produk';
3
4 jest.mock('next/router', () => ({
5   useRouter() {
6     return {
7       route: "/product",
8       pathname: "",
9       query: {},
10      asPath: "",
11      push: jest.fn(),
12      event: {
13        on: jest.fn(),
14        off: jest.fn()
15      },
16      isReady: true,
17    }
18  })
19 }));
20
21 describe("Product Page", () => {
22   it("renders product page correctly", () => {
23     const page = render(<TampilanProduk />);
24     expect(screen.getByTestId("title").textContent).toBe("Produk Page");
25     expect(page).toMatchSnapshot();
26   })
27 })
```

PRAKTIKUM 8 – MENANGANI UNDEFINED DATA

Jalankan npm run test:coverage maka akan muncul error

```
FAIL src/_test_/pages/product.spec.tsx
  • Product Page › renders product page correctly

TypeError: Cannot read properties of undefined (reading 'data')

   28 |       return (
   29 |         <div className="container mx-auto p-4">
>  30 |           <TampilanProduk products={isLoading ? [] : data.data} />
      |                                     ^
   31 |         </div>
   32 |       );
   33 |     };
      |     }

at data (src/pages/produk/index.tsx:30:61)
at Object.react_stack_bottom_frame (node_modules/react-dom/cjs/react-dom-client.development.js:25904:20)
at renderWithHooks (node_modules/react-dom/cjs/react-dom-client.development.js:7662:22)
at updateFunctionComponent (node_modules/react-dom/cjs/react-dom-client.development.js:10166:19)
at beginWork (node_modules/react-dom/cjs/react-dom-client.development.js:11778:18)
at runWithFiberInDEV (node_modules/react-dom/cjs/react-dom-client.development.js:874:13)
at performUnitOfWork (node_modules/react-dom/cjs/react-dom-client.development.js:17641:22)
at workLoopSync (node_modules/react-dom/cjs/react-dom-client.development.js:17460:14)
at flushSyncCallbackQueueImpl (node_modules/react-dom/cjs/react-dom-client.development.js:17350:9)
at flushSyncCallbackQueue (node_modules/react-dom/cjs/react-dom-client.development.js:17339:3)
```

Jika muncul error:

- o Cannot read properties of undefined

- o Perbaiki di komponen:

Pada file index.tsx pada folder pages/produk

```
pemograman_framework - index.tsx

28 return (
29   <div className="container mx-auto p-4">
30     <TampilanProduk products={isLoading ? [] : data?.data} />
31   </div>
32 );
```

Jalankan npm run test:coverage maka akan muncul error

```
FAIL src/_test_/pages/product.spec.tsx
  • Product Page › renders product page correctly

TypeError: Cannot read properties of undefined (reading 'length')

   17 |       <h1 className={styles.produk_title}>Daftar Produk</h1>
   18 |       <div className={styles.produk_content}>
>  19 |         {products.length > 0 ? (
      |                                ^
   20 |           <>
   21 |             {products.map((products: ProductType) => (
   22 |               <link href={`/${produk}/${products.id}`} key={products.id} className={styles.produk_content_1
      |             ten)}

at length (src/pages/produk/index.tsx:19:27)
at Object.react_stack_bottom_frame (node_modules/react-dom/cjs/react-dom-client.development.js:25904:20)
at renderWithHooks (node_modules/react-dom/cjs/react-dom-client.development.js:7662:22)
at updateFunctionComponent (node_modules/react-dom/cjs/react-dom-client.development.js:10166:19)
at beginWork (node_modules/react-dom/cjs/react-dom-client.development.js:11778:18)
at runWithFiberInDEV (node_modules/react-dom/cjs/react-dom-client.development.js:874:13)
at performUnitOfWork (node_modules/react-dom/cjs/react-dom-client.development.js:17641:22)
at workLoopSync (node_modules/react-dom/cjs/react-dom-client.development.js:17460:14)
at flushSyncCallbackQueueImpl (node_modules/react-dom/cjs/react-dom-client.development.js:17350:9)
at flushSyncCallbackQueue (node_modules/react-dom/cjs/react-dom-client.development.js:17339:3)
```

Maka Solusinya perbaiki code pada file

```
pemograman_framework - index.tsx

19 {products?.length > 0 ? (
20   <>
21     {products?.map((products: ProductType) => (
```

Note pastikan : comment pada code berikut pada 2 code testing

```
pemograman_framework - product.spec.tsx

24 // expect(screen.getByTestId("title").textContent).toBe("Produk Page");
```

```
pemograman_framework - about.spec.tsx

7 // expect(page.getByTestId("title").textContent).toBe("About");
```

Hasil npm run test:coverage

```
> npm run test:coverage
> my-app@0.1.0 test:coverage
> npm run test -- --coverage

> my-app@0.1.0 test
> jest --passWithNoTests -u --coverage

PASS src/_test_/pages/about.spec.tsx
PASS src/_test_/pages/product.spec.tsx

File | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files | 6.19 | 13.15 | 8.57 | 6.19 |
my-app | 0 | 0 | 0 | 0 |
next-env.d.ts | 0 | 0 | 0 | 0 | 1-6
my-app/src | 0 | 0 | 0 | 0 |
middleware.ts | 0 | 0 | 0 | 0 | 1-27
my-app/src/components/analytics | 0 | 0 | 0 | 0 |
Analytics.tsx | 0 | 0 | 0 | 0 | 1-52
my-app/src/components/layouts/Appshell | 0 | 0 | 0 | 0 |
index.tsx | 0 | 0 | 0 | 0 | 1-30
my-app/src/components/layouts/navbar | 0 | 0 | 0 | 0 |
index.tsx | 0 | 0 | 0 | 0 | 1-55
my-app/src/middleware | 0 | 0 | 0 | 0 |
withAuth.ts | 0 | 0 | 0 | 0 | 1-48
my-app/src/pages | 0 | 0 | 0 | 0 |
404.tsx | 0 | 0 | 0 | 0 | 1-26
app.tsx | 0 | 0 | 0 | 0 | 1-19
_document.tsx | 0 | 0 | 0 | 0 | 1-13
index.tsx | 0 | 0 | 0 | 0 | 1-21
my-app/src/pages/about | 100 | 100 | 100 | 100 |
index.tsx | 100 | 100 | 100 | 100 |
my-app/src/pages/admin | 0 | 0 | 0 | 0 |
index.tsx | 0 | 0 | 0 | 0 | 1-16
my-app/src/pages/blog | 0 | 0 | 0 | 0 |
[slug].tsx | 0 | 0 | 0 | 0 | 1-15
index.tsx | 0 | 0 | 0 | 0 | 1-9
my-app/src/pages/category | 0 | 0 | 0 | 0 |
[...slug].tsx | 0 | 0 | 0 | 0 | 1-18
my-app/src/pages/editor | 0 | 0 | 0 | 0 |
index.tsx | 0 | 0 | 0 | 0 | 1-13
my-app/src/pages/lapor | 0.98 | 0 | 0 | 0.98 |
[id].tsx | 0 | 0 | 0 | 0 | 1-17
index.tsx | 3.12 | 0 | 0 | 3.12 | 1-21
server.tsx | 0 | 0 | 0 | 0 | 1-25
static.tsx | 0 | 0 | 0 | 0 | 1-28
my-app/src/pages/produk | 39.08 | 50 | 33.33 | 39.08 |
index.tsx | 100 | 75 | 100 | 100 | 30
server.tsx | 0 | 0 | 0 | 0 | 1-25
static.tsx | 0 | 0 | 0 | 0 | 1-28
my-app/src/pages/produk/[produk] | 1 | 0 | 0 | 1 |
index.tsx | 0 | 0 | 0 | 0 | 1-20
server.tsx | 0 | 0 | 0 | 0 | 1-29
static.tsx | 1.96 | 0 | 0 | 1.96 | 1-50
my-app/src/pages/profil | 0 | 0 | 0 | 0 |
edit.tsx | 0 | 0 | 0 | 0 | 1-14
index.tsx | 0 | 0 | 0 | 0 | 1-13
my-app/src/pages/setting | 0 | 0 | 0 | 0 |
app.tsx | 0 | 0 | 0 | 0 | 1-9
my-app/src/pages/shop | 0 | 0 | 0 | 0 |
[...slug].tsx | 0 | 0 | 0 | 0 | 1-15
my-app/src/pages/user | 0 | 0 | 0 | 0 |
index.tsx | 0 | 0 | 0 | 0 | 1-9
my-app/src/pages/user/password | 0 | 0 | 0 | 0 |
index.tsx | 0 | 0 | 0 | 0 | 1-9
my-app/src/utills | 0 | 0 | 0 | 0 |
firebase.ts | 0 | 0 | 0 | 0 | 1-19
my-app/src/utills/db | 0 | 0 | 0 | 0 |
serviceofirebase.ts | 0 | 0 | 0 | 0 | 1-139
my-app/src/utills/sar | 100 | 100 | 100 | 100 |
fetcher.ts | 100 | 100 | 100 | 100 |

Test Suites: 2 passed, 2 total
Tests: 2 passed, 2 total
Snapshots: 2 passed, 2 total
Time: 2.153 s
Ran all test suites.
```

ANALISIS COVERAGE



Jika dilihat aplikasi yang dibuat masih 5% jadi perlu banyak perbaikan sebelum proses deploy.

Perhatikan bagian:

- Statement
- Branch
- Function
- Lines

Contoh:

Metric	Hasil
Statements	85%
Branch	60%
Functions	90%
Lines	88%

Branch biasanya paling sulit karena perlu menguji kondisi if/else.

STANDAR INDUSTRI

Biasanya:

- $\geq 80\%$ $\rightarrow$ boleh production

```

1 import { render, screen } from '@testing-library/react';
2 import TampilanProduk from '@pages/produk';
3
4 jest.mock('next/router', () => ({
5   useRouter() {
6     return {
7       route: "/product",
8       pathname: "",
9       query: {},
10      asPath: "",
11      push: jest.fn(),
12      event: {
13        on: jest.fn(),
14        off: jest.fn()
15      },
16      isReady: true,
17    }
18  })
19 }));
20
21 describe("Product Page", () => {
22   it("renders product page correctly", () => {
23     const page = render(<TampilanProduk />)
24     expect(screen.getByTestId("title").textContent).toBe("Produk Page");
25     expect(page).toMatchSnapshot();
26   })
27 })

```

```

1 import { render, screen } from '@testing-library/react';
2 import TampilanHeroProduk from '@views/produk/hero';
3
4 describe('TampilanHeroProduk', () => {
5   it('renders hero title correctly', () => {
6     const page = render(<TampilanHeroProduk />);
7     expect(screen.getByText('Produk Page')).textContent.toBe('Produk Page');
8     expect(page).toMatchSnapshot();
9   });
10 });
11

```

```
PASS src/__test__/components/produk-hero.spec.tsx
PASS src/__test__/pages/product.spec.tsx
PASS src/__test__/pages/about.spec.tsx
-----
Test Suites: 3 passed, 3 total
Tests:      3 passed, 3 total
Snapshots: 3 passed, 3 total
Time:       2.637 s
Ran all test suites.
```


2. Gunakan minimal:

- o 1 Snapshot test

- o 1 toBe()

- o 1 getByTestId()

About

```
● ● ● pemograman_framework - index.tsx
11 <h1 data-testid="title">Ini Adalah Halaman About</h1>
```

Unit test about

```
● ● ● pemograman_framework - about.spec.tsx
7 expect(page.getByTestId("title").textContent).toBe("Ini Adalah Halaman About");
8 expect(page).toMatchSnapshot();
```

Halaman product

```
● ● ● pemograman_framework - index.tsx
17 <h1 data-testid="title" className={styles.produk__title}>Daftar Produk</h1>
```

Unit test product

```
● ● ● pemograman_framework - product.spec.tsx
24 expect(screen.getByTestId("title").textContent).toBe("Daftar Produk");
25 expect(page).toMatchSnapshot();
```

Komponen Hero

```
● ● ● pemograman_framework - hero.tsx
6 <div data-testid="hero-title" className="bg-gray-100 p-5 text-center text-2xl font-bold">
```

Unit test hero

```
● ● ● pemograman_framework - produk-hero.spec.tsx
7 expect(screen.getByTestId("hero-title").textContent).toBe('Produk Page');
8 expect(page).toMatchSnapshot();
```

Hasil

```
PASS src/__test__/components/produk-hero.spec.tsx
PASS src/__test__/pages/product.spec.tsx
PASS src/__test__/pages/about.spec.tsx
```

```
Test Suites: 3 passed, 3 total
Tests:       3 passed, 3 total
Snapshots:  3 passed, 3 total
Time:        2.709 s
Ran all test suites.
```

3. Buat coverage minimal 50%

Modifikasi jest.config.mjs

```
pemograman_framework -  
  
1 import nextJest from 'next/jest.js';  
2  
3 const createJestConfig = nextJest({  
4   dir: './',  
5 })  
6  
7 const config = {  
8   coverageProvider: 'v8',  
9   testEnvironment: 'jsdom',  
10  modulePaths: ['<rootDir>/src/'],  
11  collectCoverage: true,  
12  collectCoverageFrom: [  
13    'src/pages/produk/index.tsx',  
14    'src/views/product/index.tsx',  
15    'src/views/produk/hero.tsx',  
16    'src/views/produk/index.tsx',  
17    'src/views/produk/main.tsx',  
18  ],  
19  coverageThreshold: {  
20    global: {  
21      branches: 50,  
22      functions: 50,  
23      lines: 50,  
24      statements: 50,  
25    },  
26  },  
27 }  
28  
29 export default createJestConfig(config);
```

Hasil

```
> npm run test:coverage  
> my-app@0.1.0 test:coverage  
> npm run test -- --coverage  
  
> my-app@0.1.0 test  
> jest --passWithNoTests -u --coverage  
  
PASS src/__test__/components/produk-hero.spec.tsx  
PASS src/__test__/pages/product.spec.tsx  
PASS src/__test__/pages/about.spec.tsx  


| File          | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s |
|---------------|---------|----------|---------|---------|-------------------|
| All files     | 62.68   | 63.63    | 60      | 62.68   |                   |
| pages/produk  | 100     | 75       | 100     | 100     |                   |
| index.tsx     | 100     | 75       | 100     | 100     | 30                |
| views/product | 68.62   | 75       | 100     | 68.62   |                   |
| index.tsx     | 68.62   | 75       | 100     | 68.62   | 20-35             |
| views/produk  | 30.61   | 33.33    | 33.33   | 30.61   |                   |
| hero.tsx      | 100     | 100      | 100     | 100     |                   |
| index.tsx     | 0       | 0        | 0       | 0       | 1-14              |
| main.tsx      | 0       | 0        | 0       | 0       | 1-20              |

  
Test Suites: 3 passed, 3 total  
Tests: 3 passed, 3 total  
Snapshots: 3 passed, 3 total  
Time: 2.575 s  
Ran all test suites.  
> █
```

All files

62.68% Statements 36/58 63.63% Branches 3/5 60% Functions 3/5 62.68% Lines 36/58

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter

File	Statements	Branches	Functions	Lines
pages/produk	100%	34/34	75%	34
views/produk	68.62%	35/51	75%	34
views/produkA	30.61%	15/49	33.33%	13

4. Lakukan mocking untuk router

Unit test halaman produk

```

    pemograman_framework -

4  jest.mock('next/router', () => ({
5    useRouter() {
6      return {
7        route: "/product",
8        pathname: "",
9        query: {},
10       asPath: "",
11       push: jest.fn(),
12       event: {
13         on: jest.fn(),
14         off: jest.fn()
15       },
16       isReady: true,
17     }
18   })
19 }
  
```

5. Dokumentasikan hasil coverage

```

> npm run test:coverage
> my-app@0.1.0 test:coverage
> npm run test -- --coverage

> my-app@0.1.0 test
> jest --passWithNoTests -u --coverage

PASS src/_test_/components/produk-hero.spec.tsx
PASS src/_test_/pages/product.spec.tsx
PASS src/_test_/pages/about.spec.tsx
  
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	62.68	63.63	60	62.68	
pages/produk	100	75	100	100	
index.tsx	100	75	100	100	30
views/product	68.62	75	100	68.62	
index.tsx	68.62	75	100	68.62	20-35
views/produk	30.61	33.33	33.33	30.61	
hero.tsx	100	100	100	100	
index.tsx	0	0	0	0	1-14
main.tsx	0	0	0	0	1-20

```

Test Suites: 3 passed, 3 total
Tests:       3 passed, 3 total
Snapshots:  3 passed, 3 total
Time:        2.575 s
Ran all test suites.
  
```

All files

62.68% Statements 36/58 63.63% Branches 3/5 60% Functions 3/5 62.68% Lines 36/58

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter

File	Statements	Branches	Functions	Lines
pages/produk	100%	34/34	75%	34
views/produk	68.62%	35/51	75%	34
views/produkA	30.61%	15/49	33.33%	13

Note: Setelah saya modifikasi pada tugas nomor 4.

DISKUSI & REFLEKSI

1. Mengapa unit testing penting sebelum production?

Membantu menangkap bug lebih awal, menjaga stabilitas fitur, dan mencegah regresi saat ada perubahan kode.

2. Mengapa branch coverage sulit mencapai 100%?

Karena tidak semua jalur logika mudah dipicu, terutama kondisi edge case, error path, dan skenario yang bergantung environment tertentu.

3. Apa itu mocking?

Teknik mengganti dependensi asli (seperti API, router, atau database) dengan versi tiruan agar test fokus pada unit yang diuji.

4. Kapan snapshot test digunakan?

Snapshot test digunakan saat ingin memverifikasi output render UI tetap konsisten dari waktu ke waktu dan mendeteksi perubahan tampilan yang tidak disengaja.

5. Apakah semua file harus dites?

Tidak semua file harus dites secara langsung, tetapi file yang berisi logika penting, alur bisnis, dan komponen kritis sebaiknya memiliki cakupan test yang memadai.

KESIMPULAN

Dalam praktikum ini mahasiswa telah:

- ✓ Menginstal dan mengkonfigurasi Jest
- ✓ Menggunakan React Testing Library
- ✓ Membuat unit test pada pages
- ✓ Menghasilkan coverage report
- ✓ Melakukan mocking router
- ✓ Memahami pentingnya testing di industri